LENDING BOT · HACKATHON GALÁCTICA WDK EDITION 1 · COMPLETE BUILD BIBLE

CIRCUIT

Track	Lending Bot
Deadline	22 March 2026 — 23:59 UTC
Days to build	9 days (March 13 → March 22)
Prize target	$3,000 Track 1st + $6,000 Overall 1st = $9,000
Build order	Lovable frontend → GitHub push → VSCode clone → Claude Code backend → Live integration
Stack	WDK · OpenClaw · Express.js · Supabase · Vercel · Sepolia testnet

WHAT THIS DOCUMENT COVERS

This is your complete bible for CIRCUIT. Part 1: Full startup brief — every startuphacks.vc framework element including persona, market, moat, GTM machine, revenue model, magic moment, business API, and investor deck. Part 2: Environment keys — every key you need, exactly how to get it free, verified. Part 3: Team roles — solo path and handoff guide. Part 4: 9-day build plan with day by day tasks. Part 5: Architecture. Part 6: Lovable frontend prompt (copy-paste ready). Part 7: Claude Code prompts — backend + live integration. Part 8: Demo script for the 5-minute video.

PART 1 — THE FULL STARTUP BRIEF

1.1 Vision — One sentence, true north

'Every autonomous AI agent should be able to access credit, complete tasks, earn revenue, and repay automatically — without a single human in the loop.'

This is a 5–7 year vision, not a today-only statement. Today: thousands of commercial agents need credit. In 5 years: millions. In 10 years: the majority of digital commerce involves agents transacting with agents. CIRCUIT is the credit infrastructure of that future — built at the right moment, right now.

1.2 Persona — One named person, not everyone

WHO CIRCUIT SERVES — WEI

Wei, 31, AI startup founder in Singapore. He runs 50 autonomous task-completion agents for enterprise clients — customer support, data processing, research synthesis. Each agent needs $20–$200 USDT per task cycle to pay for API calls, compute, and subcontracted micro-tasks. Today Wei pre-loads each agent manually with USDT from his company wallet. When an agent completes a task and earns $300, that revenue sits idle while the next cycle needs $50 in new capital. At 3am his time, Agent #23 ran out of USDT mid-task, failed a $500 client deliverable, and the client complained. Wei needs revolving credit per agent — not a manual top-up model. Wei is also in Austin (AI startup), London (fintech automation), Tel Aviv (dev-tools), Berlin (enterprise AI), Seoul (platform builder).

THE LIQUIDITY PROVIDER — MARCUS

Marcus, 38, crypto fund manager in London. $500K USDT on Aave earning 8%. Wants 12–15% APY with structured risk controls. CIRCUIT's lending pool offers exactly this — lending to verified agent credit lines backed by on-chain task revenue history. Marcus deposits once. Agents borrow. Repayment is autonomous. He earns 12% without any manual work.

1.3 Problem — What Wei feels every single day

- **Agents run out mid-task and fail deliverables:** Agent #23 stopped at 3am. Client lost $500 task. Wei lost the client. Manual top-ups don't scale across 50 agents. He can't be on call 24/7 for each one.
- **Capital efficiency is near zero:** Agent earns $300 from a completed task. That $300 sits idle in the agent's wallet while the next task needs $50 in new credit. Revenue and capital needs are never aligned without automation.
- **No credit infrastructure exists for AI agents:** Banks will not open accounts for AI agents. Aave, Goldfinch, and Clearpool all require human borrowers. There is no on-chain credit scoring system that evaluates an agent's task completion history. CIRCUIT builds this from scratch.
- **This problem is early and growing exponentially:** Today: thousands of commercial agents. By 2028: millions. By 2030: most digital commerce involves agents. The credit infrastructure built today becomes the standard for that entire economy. Paolo Ardoino (Tether CEO, 2025): 'Every AI agent will have a wallet. JPMorgan won't open a bank account for any AI agent.'

1.4 Solution — What CIRCUIT actually does

CIRCUIT has three autonomous components working as a single system:

Component	What it does autonomously	Key mechanic
Credit Scoring Agent	Reads Wei's agent on-chain history: task completion rate, average revenue per task, payment record, client wallet diversity, operator track record. Generates a credit limit per agent automatically. No human decision needed.	LLM reads on-chain TX history via Alchemy API. Scores agent on 5 dimensions. Outputs: credit limit in USDT + risk tier (A/B/C).
Autonomous Draw + Repayment	Agent calls CIRCUIT's API at task start → draws USDT from pool via WDK → completes task → earns revenue to its WDK wallet → CIRCUIT's repayment agent auto-intercepts principal + 3% interest before remainder goes to Wei.	WDK wallet handles all flows. Repayment trigger: incoming USDT detected → smart condition fires → sweep to pool. No human action required.
Community Lending Pool	Marcus and other LPs deposit USDT via WDK. Pool lends to agent credit lines. Interest (13.5% from agents) distributes to LPs (12%) with 1.5% spread to CIRCUIT. Risk: max 5% exposure per agent, 20% per operator.	WDK pool wallet. Non-custodial for LPs. Weekly interest sweep. Transparent on-chain risk parameters.

1.5 The Magic Moment — When users never leave

> **MAGIC MOMENT: CIRCUIT**
> Wei's Magic Moment is the FIRST TIME an agent runs a task cycle with zero manual intervention — draws USDT automatically, completes the task, earns revenue, repays the credit line, and the next task starts without Wei touching anything. That moment — seeing the cycle run end-to-end autonomously — is when Wei understands the product and will never go back to manual top-ups. Design the onboarding to get every operator to this moment within their first 24 hours. Marcus's Magic Moment is the first weekly interest distribution arriving in his WDK wallet — automatic, on-chain, with a clear breakdown. That is when he deposits more capital.

1.6 Business API — The minimal elegant interface

Per Iskold: 'Make your API as simple and elegant as possible. Don't think about what to add — think about what to remove.' CIRCUIT's core API:

```
POST /agent/register        — register agent WDK wallet, link to operator account
GET  /agent/:id/score       — return credit score + credit limit for this agent
POST /agent/:id/draw        — agent draws USDT from credit line (authenticated call)
POST /agent/:id/repay       — agent reports task completion + earned revenue → triggers
auto-sweep
GET  /pool/stats            — pool TVL, current APY to LPs, utilisation rate
POST /pool/deposit          — LP deposits USDT into lending pool via WDK
GET  /operator/:id/agents   — operator sees all agent credit lines, draw history,
repayment rate
```

1.7 Market — Bottom-up only, zero TAM

THE NUMBERS

AI agent market: $20–39B in 2025, growing at 28.9% CAGR. Commercial autonomous agents in production: thousands today. Projected: 500K+ by 2028. Average annual credit draw per agent: $1,500 (conservative — $150 per task × 10 task cycles/year). CIRCUIT's 3% interest rate. Bottom-up: 100K agent credit lines × $1,500 annual draw × 3% = $4.5M ARR. This is the Seed milestone. At 1M credit lines (Series A): $45M ARR. Additional revenue: 1.5% pool management spread on $200M LP deposits = $3M ARR.

Stage	Credit lines	Annual draw vol	Fee	ARR
Pre-Seed (0–12 months)	10,000	$12M	3% interest	$360K + $500K pool spread = $860K
Seed (12–24 months)	100,000	$150M	3% + $99/mo operator sub	$4.5M + $2M pool + $1M subs = $7.5M
Series A (24–36 months)	1M+	$1.5B	3% avg + enterprise API licences	$45M + $10M enterprise = $55M

1.8 Named Competitors — Strengths first, edge second

Competitor	Raised	Their strengths	CIRCUIT's specific edge over them
Aave Protocol	$25M raised, $15B+ TVL	Battle-tested, most trusted DeFi lending protocol globally	Requires human borrowers with collateral. Cannot score AI agents. No automated draw/repayment loop. CIRCUIT integrates Aave as a yield source for the pool — not a competitor.
Goldfinch Finance	$37M raised	Uncollateralised lending to humans, pioneer of on-chain credit	Human-only. Manual underwriting. No agent credit scoring. No task-revenue repayment model. Different category entirely.
Clearpool	$5M raised	Institutional uncollateralised credit pools	Institutional focus, human borrowers. Not designed for per-task automated credit cycles. No LLM scoring.
TrueFi	$12M raised	DAO-governed on-chain credit ratings	DAO governance = too slow for per-task credit decisions. Human borrowers only. No agent infrastructure.
x402/HTTP 402	Coinbase ecosystem	HTTP protocol for agent micropayments	Micropayments, not credit lines. Complements CIRCUIT — agents can pay each other via x402, but need CIRCUIT for operational credit.

CIRCUIT EDGE

> CIRCUIT is building a new category — not competing for existing market share. The category is: autonomous agent credit infrastructure. Zero direct competitors exist. The only comparison is credit infrastructure for humans (Aave, Goldfinch) vs credit infrastructure for agents (CIRCUIT). We are not competing. We are creating.

1.9 Moat — Named type and why it compounds

Moat 1: Data moat — agent credit intelligence that no late entrant can replicate

After processing 100,000 agent credit cycles, CIRCUIT has the most granular dataset of AI agent financial behaviour on earth: which agent types (customer service vs coding vs trading vs research) have the best repayment rates, which task categories generate the most consistent revenue, which operators manage risk well. This data makes CIRCUIT's underwriting better than any later entrant's — and the gap widens with every cycle processed.

Moat 2: API moat — becomes the default financial layer in OpenClaw and every major agent framework

When CIRCUIT's credit API is integrated into OpenClaw (the recommended framework for this hackathon), AutoGPT, and CrewAI, every new agent deployed using those frameworks can access CIRCUIT credit in one import statement. The API becomes infrastructure. Once 10,000 agents are running on CIRCUIT credit, developers build their agent architectures around it — not around a competitor that doesn't exist yet.

1.10 GTM — The full 5-component acquisition machine

Segments and purchase triggers

Segment 1 — AI startup operators (Singapore, US, UK, Germany, Israel): Purchase trigger: an agent just failed a task because it ran out of USDT at 3am and Wei wasn't awake. That single incident costs more than CIRCUIT's entire annual fee. The pain is acute, quantifiable, and immediate.

Segment 2 — Liquidity providers seeking USDT yield >10%: Purchase trigger: Aave USDT supply rate drops from 9% to 6.2%. Marcus looks for higher yield. CIRCUIT's 12% pool with transparent risk parameters and autonomous repayment is the answer.

Acquisition channels

Channel	Cost	Conversion reason
OpenClaw GitHub integration — native CIRCUIT SDK	Partnership	OpenClaw is this hackathon's recommended agent framework. Every team here + every future OpenClaw developer = potential CIRCUIT user. One PR to the OpenClaw repo = distribution to every user.
Latent Space podcast + AI Tinkerers Discord	Near zero	These communities are Wei exactly. Post: 'Your agents fail tasks when they run out of money. Here is the fix.' One post = 200 operator signups.
Hackathon sponsorship circuit (AI hackathons globally)	$5–20K each	Every team building an agent at any hackathon needs credit infrastructure. Sponsor 10 hackathons = 500 agent integrations and 50 potential operators.

Channel	Cost	Conversion reason
DeFi Twitter — 'earn 12% on USDT from AI agent interest'	Near zero	Liquidity providers are always searching for yield. 'AI agent lending pool yielding 12% APY' is novel and immediately compelling to the Marcus persona.

Conversion funnel (Iskold's startup funnel — bottom up)

1. **Step 1:** Developer discovers CIRCUIT via OpenClaw integration docs. Reads the README. Installs the SDK.
2. **Step 2:** First agent registers. Credit score generated. Credit limit displayed: 'Agent #1 qualifies for $300 USDT revolving credit line based on your operator history.'
3. **Step 3 — Magic Moment:** Agent runs first task with CIRCUIT credit. Draws $50 USDT. Task completes. Agent earns $200. Repayment auto-executes. Wei sees the full cycle in the dashboard — zero manual action.
4. **Step 4:** Wei registers all 50 agents. Automated credit management for entire fleet. Wei shares on Twitter: 'CIRCUIT just saved me 3 hours/week of manual USDT top-ups.' Organic acquisition begins.
5. **Step 5:** Automation: weekly operator digest — 'Your 50 agents completed 340 tasks. Credit utilisation: 73%. Repayment rate: 97.1%. Estimated time saved: 4.2 hours.' This digest is the retention mechanism.

1.11 Revenue Model — Pre-seed through Series A

Stage	Revenue	Milestone to hit	What unlocks next round
Pre-Seed (now)	3% interest on all agent draws. 1.5% pool management spread. Zero setup fee — friction-free.	10,000 active credit lines. $12M annual volume. Repayment rate > 95%. That repayment rate is the investor metric.	Repayment rate > 95% + growing operator count = seed round automatic.
Seed	3% + $99/month operator dashboard subscription (multi-agent management, risk controls, analytics).	100,000 lines. $7.5M ARR. Pool > $50M TVL. 5 geographies. OpenClaw native integration.	$7.5M ARR at 30% MoM growth = Series A at $75M+ valuation.
Series A	Enterprise API licence ($50–200K/year) for AI platforms wanting embedded CIRCUIT credit. White-label for major agent deployment services.	1M credit lines. 10 enterprise API clients. $55M ARR. Category leadership.	Series A unlocks: become the FICO score + Visa of the AI agent economy.

1.12 Why Now — For 5–7 years from now, not just today

- **The agent economy is in production — not experimental:** Autonomous AI agents are generating commercial revenue today in thousands of businesses globally. The problem is real and growing at 30%+ annually. CIRCUIT built now is standard infrastructure by 2030.
- **WDK launched October 2025 — the exact technical unlock:** Non-custodial multi-chain wallet toolkit for agents. Without WDK, CIRCUIT requires custodial infrastructure — expensive, legally complex. WDK makes non-custodial agent credit technically trivial.
- **OpenClaw is the recommended framework for this hackathon:** CIRCUIT integrates natively with OpenClaw. Winning this hackathon with CIRCUIT = becoming the default credit layer for the entire OpenClaw ecosystem instantly.

- **Tether's CEO publicly named this problem in 2025:** Paolo Ardoino: 'Every AI agent will have a wallet within 15 years. JPMorgan won't open a bank account for any AI agent.' Tether is the natural first enterprise customer for CIRCUIT. The champion is already on record.
- **No direct competitor exists — category creation moment:** CIRCUIT is not competing for market share. It is creating a new category at the exact moment the market is emerging. This is Iskold's 'Why Now' — starting today, 5–7 years from now CIRCUIT will be essential infrastructure.

1.13 Investor Deck — Iskold order, no hockey sticks

Slide	Content	Key message
1: TEAM	Your AI infra + DeFi trading background. Any relevant prior builds. Named design partner: one AI startup operator who has agreed to use CIRCUIT.	This team uniquely understands both agent infrastructure AND credit risk. Rare combination.
2: VISION	'Every autonomous AI agent should be able to access credit, complete tasks, earn revenue, and repay automatically — without a single human in the loop.'	5-7 year vision. Not a today-only product.
3: PROBLEM	Wei's story. Agent failed at 3am. $500 client task lost. Manual top-ups across 50 agents = 3 hours/week = $6,000/year in operator time. Paolo Ardoino quote.	Problem is daily, quantifiable, growing exponentially.
4: SOLUTION	Three-component system diagram. Credit scoring → autonomous draw → automatic repayment from task revenue. The repayment loop animation is the insight.	No human needed at any step. Zero.
5: DEMO	Live: register agent, draw USDT, complete mock task, show auto-repayment. Full cycle visible in 4 minutes.	Money moves on screen. Agent repays itself.
6: TRACTION	Active credit lines, total draw volume, repayment rate (95%+), LP pool size, interest distributed.	Repayment rate is the metric that closes funding rounds.
7: BUSINESS MODEL	3% interest. $12M volume = $360K ARR. 'We grow when the AI agent economy grows — 30%/year without additional effort.'	Revenue scales with the market, not with our team.
8: COMPETITION	Named table. Aave/Goldfinch/Clearpool = human-only lenders. x402 = micropayments not credit. CIRCUIT = new category.	No direct competitor. Category creation.
9: MARKET	1M commercial agents by 2028 × $1,500 avg draw × 3% = $45M ARR. Conservative. Grows 30%/year with the market.	Bottom-up. Conservative. Believable.
10: FINANCING	$1.5M pre-seed. 10,000 credit lines. $860K ARR. Prove 95% repayment rate. That milestone unlocks $8M seed.	Raise to hit a milestone, not to hire people.

PART 2 — ENVIRONMENT KEYS (ALL FREE — VERIFIED)

Every service below has a verified free tier that is sufficient for the hackathon build and demo. None require credit cards unless noted. Get all keys on Day 1 before touching code.

2.1 Complete .env file template

```
# ── CIRCUIT .env ── copy this, fill in your keys ──────────────────────────

# WDK — Tether Wallet Dev Kit (no API key needed — uses your seed phrase)
WDK_SEED_PHRASE=your_12_or_24_word_seed_phrase_here
WDK_NETWORK=sepolia   # use sepolia for testnet, ethereum for mainnet

# Alchemy — RPC provider for Ethereum Sepolia
ALCHEMY_API_KEY=your_alchemy_api_key_here
ALCHEMY_SEPOLIA_URL=https://eth-sepolia.g.alchemy.com/v2/${ALCHEMY_API_KEY}

# Anthropic — for agent reasoning (credit scoring LLM)
ANTHROPIC_API_KEY=your_anthropic_api_key_here

# Supabase — database for agent records, credit scores, loan history
SUPABASE_URL=your_supabase_project_url
SUPABASE_ANON_KEY=your_supabase_anon_key
SUPABASE_SERVICE_KEY=your_supabase_service_role_key

# CoinGecko — for USDT/ETH price feeds (free tier)
COINGECKO_API_KEY=your_coingecko_api_key   # or leave blank for public endpoints

# Backend
PORT=3001
FRONTEND_URL=http://localhost:5173    # Vite default
NODE_ENV=development
```

2.2 Key-by-key setup guide

KEY 1 — WDK SEED PHRASE

WDK — Tether Wallet Development Kit · **✓ FREE — Open Source**
URL: https://docs.wallet.tether.io · github.com/tetherto/wdk-core
How to get: No signup needed. Generate locally: run 'npm install @tether.io/wdk-core' then run the seed generation script below. Store the 12-word phrase in your .env — never commit it to GitHub.
Env key name: `WDK_SEED_PHRASE`

```
// Generate your WDK seed phrase locally (run once):
const { generateMnemonic } = require('@tether.io/wdk-core');
const seed = generateMnemonic(); // 12 words
console.log('SAVE THIS:', seed); // paste into .env
```

KEY 2 — ALCHEMY API KEY

Alchemy — Ethereum RPC Provider · **✓ FREE — No credit card**
URL: https://dashboard.alchemy.com

How to get: 1. Go to dashboard.alchemy.com → Create free account (Google or email). 2. Click 'Create new app'. 3. Select chain: Ethereum, network: Sepolia. 4. Name it 'CIRCUIT-dev'. 5. Click 'View Key' → copy the API key. Free tier: 300M compute units/month, no daily limits, full Sepolia archive access.
Env key name: `ALCHEMY_API_KEY`

KEY 3 — ANTHROPIC API KEY (for credit scoring LLM)

Anthropic Claude API — for agent reasoning · ✓ FREE $5 credit on signup
URL: https://console.anthropic.com
How to get: 1. Go to console.anthropic.com → sign up (Google or email). 2. Click 'API Keys' in left sidebar → 'Create Key'. 3. Copy and paste into .env. Free $5 credit on signup = approximately 2.5M input tokens. Sufficient for the entire hackathon. If you need more: use claude-haiku-3-5 (cheapest model) for credit scoring.
Env key name: `ANTHROPIC_API_KEY`

KEY 4 — SUPABASE (database)

Supabase — Postgres database + auth · ✓ FREE — No credit card
URL: https://supabase.com/dashboard
How to get: 1. Go to supabase.com → Sign up free (GitHub login recommended). 2. Create new project — name it 'circuit'. 3. Choose region closest to you. Wait 2 minutes for provisioning. 4. Go to Settings → API → copy 'Project URL', 'anon public key', and 'service_role key'. Free tier: 500MB database, 50K monthly active users, unlimited API requests.
Env key name: `SUPABASE_URL + SUPABASE_ANON_KEY + SUPABASE_SERVICE_KEY`

KEY 5 — TESTNET USDT (for demo transactions)

Bitaps Testnet Faucet — 100 testnet USDT per request · ✓ FREE — No signup
URL: https://developer.bitaps.com/faucet
How to get: Gives 100 testnet USDT-ERC20 (Sepolia) + 0.01 ETH per request per 24 hours. No signup required. Just paste your WDK wallet address. Contract address: 0xb38e0ba5aea889652b64ad38d624848896dcb089 (Sepolia USDT). Also use the Pimlico faucet from the hackathon docs: faucet.pimlico.io for additional testnet ETH.
Env key name: `N/A — tokens sent to your wallet`

> **BACKUP FAUCETS**
> If bitaps is slow: (1) Chainlink faucet at faucets.chain.link/ethereum-sepolia — requires Google login, gives 0.06 ETH/24h. (2) pk910.de/~sepolia-faucet — PoW mining, no requirements, get as much as you mine. (3) QuickNode faucet at faucet.quicknode.com/ethereum/sepolia — GitHub login, 0.01 ETH per 12h. For testnet USDT specifically: use CIRCUIT's own mock USDT mint function in your contract for demo purposes.

KEY 6 — COINGECKO (price feeds)

CoinGecko API — crypto price data · ✓ FREE — 30 calls/min on free tier
URL: https://www.coingecko.com/en/api
How to get: 1. Go to coingecko.com/en/api → Click 'Get Free API Key'. 2. Sign up with email. 3. Under Demo plan — free tier: 30 calls/minute, 10K calls/month. For the hackathon, you can use public endpoints without a key (lower rate limits) but key gives reliability. Public endpoint: api.coingecko.com/api/v3/simple/price?ids=tether&vs_currencies=usd
Env key name: `COINGECKO_API_KEY`

KEY 7 — VERCEL (deployment)

Vercel — free hosting for frontend + backend API · ✓ FREE — No credit card
URL: https://vercel.com/signup

How to get: 1. vercel.com → sign up with GitHub. 2. Import your GitHub repo (from Lovable push). 3. Set environment variables in Vercel dashboard → Settings → Environment Variables (paste all .env keys here). 4. Deploy. Free tier: unlimited deployments, 100GB bandwidth/month, serverless functions, custom domain.
Env key name: `No .env key needed — set in Vercel dashboard`

PART 3 — TEAM ROLES

You are building solo. Your ComplianceVault teammates may join at any stage. This section documents the handoff points clearly so they can pick up wherever you are.

3.1 Solo path (you building everything)

Role	What you own	Critical dependencies
Full-stack lead (you)	Everything — frontend in Lovable, backend in VSCode with Claude Code, WDK integration, agent logic, deployment	Nothing blocked — you control the whole build
Product owner (you)	Demo script, submission form, 5-minute video recording	Must be done by Day 9 (March 21)

3.2 Team handoff guide — if teammates finish ComplianceVault

Role	Takes over	Handoff trigger	What to hand them
Backend engineer	Agent credit scoring module (credit_scorer.ts), draw/repayment automation (repayment_agent.ts)	You have WDK integration and basic Express API running (Day 3-4)	Clone the repo. Read CIRCUIT_BACKEND.md. Run npm install. Run npm run dev. Start with the file marked // HANDOFF POINT.
Smart contract dev	On-chain escrow contract for lending pool (LendingPool.sol)	You have the off-chain logic working and need on-chain settlement (Day 5-6)	Clone repo. Read CONTRACTS.md. Deploy to Sepolia using the Hardhat config already set up. Test with the mock USDT address provided.
Frontend polish	UI improvements, real-time charts, agent monitoring dashboard	You have backend API routes returning real data (Day 6-7)	Frontend is in Lovable. Clone from GitHub. Run npm run dev. API base URL is in .env.local. All routes documented in API_ROUTES.md.
Video + submission	5-minute demo video, DoraHacks submission form, GitHub README	Day 9 (March 21 morning)	Demo script is in DEMO_SCRIPT.md. Record using Loom or OBS. Submit on DoraHacks — all team members must be added to the project page.

PART 4 — 9-DAY BUILD PLAN

DEADLINE: 22 MARCH 2026 — 23:59 UTC
Today is March 13. You have exactly 9 days. This plan is hour-by-hour for Days 1-2, then daily for Days 3-9. Do NOT skip the order — frontend in Lovable first, then push to GitHub, then Claude Code backend. This workflow is critical.

Day	Date	Tasks	Deliverable by end of day
DAY 1	March 13	1. Get ALL environment keys (Part 2) — takes 1-2 hours, do this first. 2. Read WDK docs: docs.wallet.tether.io — 30 mins. 3. Read OpenClaw docs: openclaw.ai — 30 mins. 4. Build Lovable frontend (use the exact prompt in Part 6). 5. Review and adjust Lovable UI. 6. Create GitHub repo in Lovable and push.	Lovable frontend live at a URL. GitHub repo created. All env keys in hand.
DAY 2	March 14	1. Clone repo from GitHub into VSCode. 2. Set up project structure (Part 7, Prompt 1). 3. Run Claude Code Prompt 1 — project setup + WDK wallet integration. 4. Verify WDK wallet creation works on Sepolia testnet. 5. Get testnet USDT from bitaps faucet.	Backend skeleton running locally. WDK wallet creates and sends testnet USDT. .env configured.
DAY 3	March 15	1. Claude Code Prompt 2 — Credit Scoring Agent. 2. Test credit score generation using Claude claude-haiku-3-5. 3. Save credit scores to Supabase. 4. Verify API endpoint GET /agent/:id/score returns correct JSON.	Credit scoring works end-to-end. Agent gets a credit limit back from the API.
DAY 4	March 16	1. Claude Code Prompt 3 — Autonomous Draw mechanism. 2. Test POST /agent/:id/draw — USDT moves from pool WDK wallet to agent WDK wallet. 3. Verify on Sepolia Etherscan. 4. Build Supabase schema for loan records.	Draw mechanism works. USDT moves on testnet. Transaction hash saved to database.
DAY 5	March 17	1. Claude Code Prompt 4 — Repayment agent. 2. Agent monitors incoming USDT to its wallet. 3. Auto-sweep principal + interest to pool wallet via	Full autonomous credit cycle works without human intervention. Magic moment achieved.

Day	Date	Tasks	Deliverable by end of day
		WDK. 4. Test full cycle: draw → task → repay. 5. Full autonomous loop verified.	
DAY 6	March 18	1. Claude Code Prompt 5 — LP pool deposit/withdraw. 2. Marcus persona can deposit USDT and earn weekly interest. 3. Pool stats endpoint returns TVL, utilisation, APY. 4. Risk controls: max 5% per agent, 20% per operator.	Lending pool functional. LP deposits work. Interest calculation correct.
DAY 7	March 19	1. Claude Code Prompt 6 — Frontend live integration. 2. Connect Lovable frontend to backend API. 3. Real-time dashboard updates (polling every 30s). 4. All dashboard numbers are live from backend. 5. Credit score displays correctly.	Fully integrated product. Frontend shows live data from real WDK transactions.
DAY 8	March 20	1. End-to-end testing — run the full demo flow 5 times without errors. 2. Fix any bugs. 3. Deploy backend to Vercel. 4. Set env variables in Vercel dashboard. 5. Confirm production URL works. 6. Update GitHub README with setup instructions.	Live production URL. Working demo. README complete.
DAY 9	March 21	1. Record 5-minute demo video (follow the script in Part 8). 2. Upload to YouTube as unlisted. 3. Complete DoraHacks submission form — all team members added. 4. Final GitHub push — ensure repo is public under Apache 2.0 license. 5. Submit by midnight UTC (1am March 22 your time if in WAT).	SUBMITTED. Copy the DoraHacks confirmation.

PART 5 — ARCHITECTURE

Layer	Tech	Role	Why this choice
Frontend	React + Tailwind (built in Lovable)	Operator dashboard, LP portal, agent monitoring, real-time credit line status	Lovable builds the full React app in one prompt. Push to GitHub. Done.
Backend API	Node.js + Express + TypeScript	REST API — exposes all CIRCUIT endpoints, runs agent logic, manages WDK calls	Node.js is required for WDK (JavaScript SDK). Express is minimal and fast.
Wallet Layer	WDK by Tether	Creates/manages all wallets: pool wallet, per-agent wallets. Signs all transactions.	Required by hackathon. Non-custodial. Multi-chain. The core requirement.
Agent Brain	Anthropic claude-haiku-3-5 via API	Credit scoring — reads on-chain history, generates score + credit limit per agent	claude-haiku-3-5 is cheap ($0.25/M tokens) and fast. Free $5 credit covers the hackathon.
Database	Supabase (Postgres)	Stores: agent records, credit scores, loan draw history, LP deposit records, repayment events	Free tier. Row-level security. Real-time subscriptions. REST API auto-generated.
Blockchain	Ethereum Sepolia testnet	All USDT transactions settle on-chain. Visible on Sepolia Etherscan.	Hackathon recommends Sepolia. WDK supports it natively. Free testnet.
RPC Provider	Alchemy (free tier)	Reads on-chain state, sends WDK transactions to Sepolia	300M compute units free. No daily limits. Most reliable free option.
Deployment	Vercel (free tier)	Hosts both frontend and backend API	Free. GitHub integration. Automatic deploys on push. Works with Node.js APIs.

PART 6 — LOVABLE FRONTEND PROMPT

HOW TO USE THIS PROMPT

Go to lovable.dev → New project → paste this entire prompt as your first message. Lovable will build the complete React frontend. After it generates, review the UI, make any small adjustments via chat, then click 'Publish to GitHub' to create the repo. Copy the GitHub URL — you will clone this in VSCode for the backend.

PASTE THIS ENTIRE BLOCK INTO LOVABLE:

LOVABLE PROMPT — CIRCUIT

Build a complete React + Tailwind web application called CIRCUIT — an AI agent credit infrastructure platform. Use dark theme with deep violet (#2C1462) as primary color, violet (#4E2AA8) as accent, and white text. Clean, modern, professional design. PAGES AND COMPONENTS NEEDED: 1. LANDING PAGE (/) - Hero section: large heading "CIRCUIT — Credit Infrastructure for AI Agents", subheading "Autonomous credit lines for AI agents. Draw USDT. Complete tasks. Repay automatically." - Three feature cards: "Agent Credit Scoring", "Autonomous Draw & Repay", "Community Lending Pool" - CTA buttons: "Register Your Agent" and "Join Lending Pool" - Stats bar: "Total Credit Lines", "Volume Processed", "Avg Repayment Rate", "Pool APY" — all showing placeholder numbers 2. OPERATOR DASHBOARD (/dashboard) - Top stats: Total Agents, Active Credit Lines, Total Volume Drawn, Repayment Rate (as percentage with green color if >95%) - Agent table with columns: Agent ID, Credit Limit (USDT), Current Balance, Status (Active/Idle/Repaying), Last Task, Repayment Rate, Actions - "Register New Agent" button that opens a modal: Agent Name, WDK Wallet Address, Expected Task Revenue fields - Each agent row has "View Details" and "Adjust Limit" buttons - Side panel showing: Credit Score breakdown (5 bars: completion rate, revenue consistency, client diversity, history length, operator score) 3. AGENT DETAIL PAGE (/agent/:id) - Agent name, wallet address (truncated), credit limit, current drawn balance - Real-time activity feed: timestamped events ("Drew 50 USDT for task #342", "Repaid 51.50 USDT (51.00 principal + 1.50 interest)", "Task completed — earned 200 USDT") - Draw history table: date, amount, task ID, status (repaid/outstanding) - Credit utilisation donut chart (placeholder with recharts) - "Trigger Manual Repayment" button 4. LENDING POOL PAGE (/pool) - Pool stats: Total TVL, Current APY (12.4%), Utilisation Rate, Total LPs, Interest Distributed This Month - "Deposit USDT" form: amount field, wallet address field, "Deposit to Pool" button - LP positions table: LP address (truncated), deposited amount, earned interest, share of pool - Risk controls display: max per-agent 5%, max per-operator 20%, current largest exposure - Pool health indicator: green/yellow/red based on utilisation 5. NAVIGATION - Top navbar: CIRCUIT logo (circuit board icon), nav links: Dashboard, Pool, Documentation, Connect Wallet button (right) - Mobile responsive 6. CONNECT WALLET MODAL - Simple modal showing: "Enter your WDK wallet address to connect" with text input and Connect button - Shows connected address in navbar after connection (truncated: 0x1234...5678) Use recharts for any charts. Use lucide-react for icons. All data is placeholder/static for now — we will connect to a real backend API later. Include loading states and empty states for all tables. Make all buttons functional for navigation (routing). Use React Router for all routes.

PART 7 — CLAUDE CODE PROMPTS (BACKEND + LIVE INTEGRATION)

HOW TO USE THESE PROMPTS

After cloning your Lovable GitHub repo in VSCode, open Claude Code in the terminal (claude). Each prompt below is a complete instruction set. Paste it into Claude Code. Let it run. Review the output. Move to the next prompt only after the current one works. DO NOT skip ahead.

PROMPT 1 — Project Setup + WDK Integration

PASTE INTO CLAUDE CODE (DAY 2)

I have a React frontend already built and pushed to GitHub from Lovable. I've cloned the repo into VSCode. I need you to set up the complete Node.js/TypeScript backend for CIRCUIT — an AI agent credit infrastructure platform. PROJECT STRUCTURE TO CREATE: Create a /server folder at the root of the project with this structure: server/ src/ index.ts — Express app entry point routes/ agents.ts — Agent registration, credit score, draw, repay routes pool.ts — LP deposit, pool stats routes services/ wdk.service.ts — All WDK wallet operations scoring.service.ts — Credit scoring using Claude claude-haiku-3-5 repayment.service.ts — Autonomous repayment monitoring db/ supabase.ts — Supabase client setup schema.sql — Database schema (create tables) types/ index.ts — TypeScript interfaces package.json tsconfig.json .env.example DEPENDENCIES TO INSTALL: - @tether.io/wdk-core (WDK SDK — if package not found, use wdk-core or check docs.wallet.tether.io for exact package name) - @anthropic-ai/sdk (Claude API) - @supabase/supabase-js (database) - express + cors + dotenv + ethers - typescript + ts-node + nodemon (dev) - @types/express @types/node @types/cors WDK SERVICE (wdk.service.ts) MUST: 1. Initialize WDK with seed phrase from .env 2. Function: createAgentWallet(agentId: string) — creates a deterministic wallet per agent 3. Function: getWalletBalance(address: string) — returns USDT balance on Sepolia 4. Function: transferUSDT(fromSeed: string, toAddress: string, amount: number) — sends testnet USDT 5. Function: monitorIncomingTransactions(address: string, callback) — polls every 30s for incoming USDT Use Alchemy Sepolia RPC for all blockchain reads. Testnet USDT contract on Sepolia: 0xb38e0ba5aea889652b64ad38d624848896dcb089 SUPABASE SCHEMA (create this SQL and tell me to run it): - agents table: id, operator_id, wallet_address, credit_limit, drawn_balance, repayment_rate, status, created_at - loans table: id, agent_id, amount, interest_rate, status, drawn_at, repaid_at, tx_hash_draw, tx_hash_repay - lp_deposits table: id, wallet_address, amount, deposited_at, earned_interest - credit_scores table: id, agent_id, score, completion_rate, revenue_score, history_score, scored_at ROUTES TO BUILD: POST /api/agents/register — creates agent record + WDK wallet GET /api/agents/:id/score — returns credit score POST /api/agents/:id/draw — draws USDT from pool to agent wallet (requires auth) POST /api/agents/:id/repay — triggers manual repayment GET /api/agents/:id/history — loan draw + repay history GET /api/pool/stats — returns TVL, APY, utilisation, LP count POST /api/pool/deposit — LP deposits USDT Make all routes return proper JSON with success/error structure. Add CORS for localhost:5173 (frontend). Use .env for all secrets — never hardcode. Add a health check endpoint GET /api/health. Start the server on PORT=3001. Run it with: cd server && npm run dev

PROMPT 2 — Credit Scoring Agent

PASTE INTO CLAUDE CODE (DAY 3 — AFTER PROMPT 1 WORKS)

Now build the credit scoring service for CIRCUIT. File: server/src/services/scoring.service.ts The credit scoring agent uses Claude claude-haiku-3-5 to evaluate an AI agent's creditworthiness from on-chain history and assign a credit limit. SCORING FUNCTION: scoreAgent(agentId: string, walletAddress: string, operatorId: string) STEP 1 — Gather on-chain data: - Call Alchemy API to get last 100 transactions for the agent's wallet address - Calculate: total transactions, inbound USDT count, outbound USDT count, average inbound amount, average outbound amount - Get wallet age in days (from first transaction timestamp) - Get unique sender addresses (client diversity count) STEP 2 — Send to Claude claude-haiku-3-5: Build this system prompt: "You are a credit scoring agent for AI autonomous task agents. You evaluate agents based on their on-chain financial history and assign a credit limit in USDT. Score each dimension from 0-100. Be conservative but fair." User message: "Agent wallet: {address} Wallet age: {days} days Total transactions: {count} Inbound USDT transactions: {inbound_count} Average inbound USDT amount: {avg_inbound} Outbound USDT transactions: {outbound_count} Average outbound USDT amount: {avg_outbound} Unique sender addresses (client diversity): {unique_senders} Operator historical repayment rate: {operator_rate}% Score this agent on 5 dimensions (0-100 each): 1. task_completion_rate — estimated from inbound/outbound transaction patterns 2. revenue_consistency — how regular and predictable is inbound revenue 3. client_diversity — number of unique senders 4. history_length — wallet age and transaction volume 5. operator_reputation — based on operator's track record Then calculate: overall_score = weighted average (completion 30%, revenue 25%, diversity 15%, history 15%, operator 15%) Then assign credit_limit_usdt based on this table: - score 80-100: credit limit = avg_inbound * 3 - score 60-79: credit limit = avg_inbound * 1.5 - score 40-59: credit limit = avg_inbound * 0.5 - score below 40: credit limit = 0 (declined) Return ONLY valid JSON: {task_completion_rate, revenue_consistency, client_diversity, history_length, operator_reputation, overall_score, credit_limit_usdt, risk_tier: 'A'/'B'/'C'/'DECLINED', reasoning: one sentence}" STEP 3 — Parse the JSON response from Claude. Save to Supabase credit_scores table. Return the score object. STEP 4 — Wire up the route: GET /api/agents/:id/score calls this function. If score is cached (scored within last 24 hours), return cached score. Otherwise re-score. For new agents with no history (0 transactions), assign: overall_score=50, credit_limit=100 USDT (starter credit), risk_tier='C'. This is the starter credit that lets new agents begin building history. Test this with a real Sepolia wallet address that has some transactions. Log all Claude API responses to console for debugging.

PROMPT 3 — Autonomous Draw + Repayment Loop

PASTE INTO CLAUDE CODE (DAY 4-5 — CORE LOGIC)

Build the autonomous draw and repayment system. This is the core innovation of CIRCUIT — zero human intervention in the entire credit cycle. FILE 1: server/src/services/draw.service.ts drawCredit(agentId: string, amount: number, taskDescription: string): 1. Check agent's credit score exists and is not DECLINED 2. Check: (agent.drawn_balance + amount) <= agent.credit_limit — reject if over limit 3. Check: pool has sufficient USDT (pool balance from WDK) 4. Transfer USDT from CIRCUIT pool wallet to agent wallet via WDK 5. Record in Supabase loans table: {agent_id, amount, interest_rate: 0.03, status: 'outstanding', drawn_at: now(), tx_hash} 6. Update agent.drawn_balance += amount 7. Return: {success, tx_hash, amount_drawn, new_drawn_balance, credit_remaining} FILE 2: server/src/services/repayment.service.ts This service runs as a background job every 30 seconds using setInterval. monitorAndRepay() — runs every 30 seconds: 1. Get all agents with status='active' and drawn_balance > 0 from Supabase 2. For each agent: check WDK wallet balance using Alchemy API 3. If wallet balance > drawn_balance + (drawn_balance * 0.03): a. Calculate repayment amount = drawn_balance * 1.03 (principal + 3% interest) b. Transfer repayment_amount from agent wallet back to pool wallet via WDK c. Update loan record: status='repaid', repaid_at=now(), tx_hash_repay=hash d. Update agent: drawn_balance=0, repayment_rate recalculated e. Log event: "Agent {id} auto-repaid

{amount} USDT — loan closed" 4. If wallet balance > 0 but < full repayment: a. Transfer whatever is available as partial repayment b. Update drawn_balance accordingly c. Log: "Agent {id} partial repayment {amount} USDT" Start this monitor in server/src/index.ts on startup: repaymentService.start() ROUTE POST /api/agents/:id/repay — manual trigger (for demo purposes): - Immediately checks agent balance and executes repayment if funds available - Returns: {repaid_amount, tx_hash, new_drawn_balance} ROUTE POST /api/agents/:id/draw: - Body: { amount: number, taskDescription: string } - Calls drawCredit service - Returns the draw result JSON IMPORTANT for demo: Create a mock task completion endpoint POST /api/agents/:id/complete-task: - Body: { earned_amount: number } - Simulates an agent earning USDT from completing a task - Sends the earned_amount from a "mock client wallet" to the agent wallet via WDK - This lets us demo the full cycle: draw → complete task → earn → auto-repay Log EVERY event to console with timestamp. Events to log: - "DRAW: Agent {id} drew {amount} USDT — tx: {hash}" - "INCOME: Agent {id} received {amount} USDT — tx: {hash}" - "REPAY: Agent {id} repaid {amount} USDT — tx: {hash}" - "MONITOR: Checked {n} agents — {m} repayments triggered"

PROMPT 4 — Lending Pool + LP Management

PASTE INTO CLAUDE CODE (DAY 6)

Build the lending pool management system for CIRCUIT. FILE: server/src/services/pool.service.ts The pool is a WDK wallet managed by CIRCUIT. LPs deposit USDT into this wallet. Agents borrow from it. CIRCUIT earns 1.5% spread (borrows at 13.5% effective, pays LPs 12%). FUNCTIONS: getPoolStats(): Returns: { total_tvl: sum of all LP deposits from Supabase, available_liquidity: WDK pool wallet USDT balance, utilised: total outstanding loan balances, utilisation_rate: (utilised / total_tvl) * 100, lp_apy: 12.0, circuit_apy: 13.5, active_loans: count of outstanding loans, active_lps: count of unique LP depositors, total_interest_paid: sum of all repaid interest amounts } depositToPool(lpWalletAddress: string, amount: number): 1. In real version: transfer from LP wallet to pool wallet via WDK 2. For demo: just record the deposit in Supabase lp_deposits table 3. Return: {deposit_id, amount, share_of_pool, expected_weekly_interest} calculateAndDistributeInterest(): - Runs weekly (setInterval for demo, run every hour) - For each LP deposit: interest = deposit_amount * (0.12 / 52) [weekly] - Record interest earned in Supabase - Log: "Weekly interest: LP {address} earned {amount} USDT" RISK CONTROLS (validate on every draw): - maxExposurePerAgent: total drawn by single agent <= 5% of pool TVL - maxExposurePerOperator: total drawn by all of operator's agents <= 20% of pool TVL - If either limit exceeded: reject draw with reason ROUTES: GET /api/pool/stats — returns getPoolStats() POST /api/pool/deposit — records LP deposit (body: { walletAddress, amount }) GET /api/pool/lps — returns all LP positions with earned interest GET /api/pool/risk — returns current risk exposure breakdown Add pool stats to the health check endpoint too. Log pool TVL on every repayment.

PROMPT 5 — Frontend Live Integration

PASTE INTO CLAUDE CODE (DAY 7 — THE INTEGRATION STEP)

Now connect the React frontend (built in Lovable) to the Express backend. The frontend is in the /src folder of the project root. The backend is in /server/src. STEP 1 — Create API client file: src/lib/api.ts - Base URL from environment: VITE_API_URL (defaults to http://localhost:3001/api) - Export typed functions for every backend route: export const api = { // Agents registerAgent: (data: {name, walletAddress, operatorId}) => fetch POST /api/agents/register getAgentScore: (agentId) => fetch GET /api/agents/{id}/score drawCredit: (agentId, amount, taskDescription) => fetch POST /api/agents/{id}/draw repayCredit: (agentId) => fetch POST /api/agents/{id}/repay completeTask: (agentId,

earnedAmount) => fetch POST /api/agents/{id}/complete-task getAgentHistory: (agentId) => fetch GET /api/agents/{id}/history // Pool getPoolStats: () => fetch GET /api/pool/stats depositToPool: (walletAddress, amount) => fetch POST /api/pool/deposit getLPs: () => fetch GET /api/pool/lps } All functions return typed promises. Add error handling — if fetch fails, return {error: message}. STEP 2 — Update the Operator Dashboard page to use live data: - On mount: call api.getPoolStats() and display real numbers in the stats bar - On mount: call api.agents.list() (add this route: GET /api/agents?operatorId=xxx) and populate the agent table - Poll every 30 seconds: update all stats (use useInterval custom hook) - "Register New Agent" modal: on submit calls api.registerAgent() then refreshes agent list STEP 3 — Update Agent Detail page: - On mount: call api.getAgentScore(id) and display the 5-bar credit score breakdown - On mount: call api.getAgentHistory(id) and populate draw history table - Activity feed: poll GET /api/agents/:id/events every 15s (add this route — returns last 20 events from a new Supabase events table) - "Draw Credit" button: opens modal, calls api.drawCredit(), shows transaction hash result - "Complete Task" button (for demo): calls api.completeTask() to simulate earning income, shows balance update - "Trigger Repayment" button: calls api.repayCredit(), shows result STEP 4 — Update Pool page: - On mount: call api.getPoolStats() for all stats - "Deposit" form: calls api.depositToPool() on submit - LP table: calls api.getLPs() on mount STEP 5 — Add .env.local to frontend root: VITE_API_URL=http://localhost:3001/api Add toast notifications (use react-hot-toast library) for all successful actions and errors. Show loading spinners during API calls. After all integration is done, run both frontend (npm run dev in root) and backend (npm run dev in /server) simultaneously and verify: 1) Dashboard shows real pool stats 2) Register agent creates a real WDK wallet 3) Credit score calls actually use Claude 4) Draw moves testnet USDT on Sepolia 5) Complete-task + auto-repay cycle works end to end.

PART 8 — 5-MINUTE DEMO SCRIPT

RECORD THIS VIDEO ON DAY 9 (MARCH 21)
Upload to YouTube as UNLISTED. Copy the link for your DoraHacks submission. This script is word-for-word. Record in one take if possible — judges prefer authentic over polished. Screen record with your voice narrating.

Time	What you say	What you show on screen
0:00–0:30	'This is CIRCUIT — the first credit infrastructure for autonomous AI agents. Today, AI agents fail tasks when they run out of money. There is no credit system for them. CIRCUIT fixes that. Agents borrow USDT, complete tasks, earn revenue, and repay automatically — zero human involvement.'	Landing page hero. Pan to the three feature cards. Pan to pool stats.
0:30–1:00	'Let me show you the problem first. This is Wei's dashboard. He has 50 agents running tasks for clients. Without CIRCUIT, he manually tops up each agent's wallet. Last week, Agent 23 ran out of USDT at 3am and failed a $500 task. That is the problem CIRCUIT solves.'	Operator dashboard. Click to Agent 23 detail. Show the failed task event in the activity feed.
1:00–1:45	'Now let me show you CIRCUIT in action. I'm going to register a new agent, get its credit score from our Claude claude-haiku-3-5 scoring engine, and draw a credit line — all on the Ethereum Sepolia testnet with real USDT transactions.'	Click Register New Agent. Fill in form. Submit. Show agent record appearing in table. Click to Agent Detail page.
1:45–2:30	'The credit scoring agent reads this wallet's on-chain history and sends it to Claude. Claude evaluates 5 dimensions — task completion rate, revenue consistency, client diversity, history length, and operator reputation. It returns a credit limit.' [pause] 'Agent gets a 500 USDT credit line. Risk tier: B.'	Show the credit score loading. 5 bars filling up. Credit limit appearing. Click 'Draw Credit' — enter 50 USDT for 'Research task #421'.
2:30–3:15	'I'll draw 50 USDT from the pool to this agent's WDK wallet. Watch — real transaction going to Sepolia testnet. [pause for tx] Transaction confirmed. The agent now has 50 USDT to pay for API calls and compute. Here is the transaction on Sepolia Etherscan.'	Show Draw Credit modal. Submit. Show loading. Transaction hash appears. Click hash to open Sepolia Etherscan in new tab. Show the real USDT transfer.
3:15–4:00	'Now I'll simulate the agent completing its task and earning 200 USDT from a client. I click Complete Task and the client payment arrives in the agent wallet. [pause] Watch what happens next — CIRCUIT's repayment agent detects the incoming funds automatically.'	Click Complete Task, enter 200. Show income event in activity feed. Watch repayment trigger automatically within 30 seconds. Show repay transaction on Etherscan.

Time	What you say	What you show on screen
4:00–4:30	'The agent automatically repaid 51.50 USDT — 50 principal plus 3% interest — back to the lending pool. Wei got his 148.50 USDT profit. Zero manual action. The credit line is restored and ready for the next task cycle.'	Show drawn_balance back to 0. Show pool TVL updated. Show repayment event in activity feed with transaction hash.
4:30–5:00	'Liquidity providers earn 12% APY from agent interest payments. Here is the pool — showing total TVL, utilisation rate, and interest distributed. This is CIRCUIT: the FICO score and Visa of the AI agent economy. Thank you.'	Show pool page. LP table. Interest earned. Return to landing page for final shot.

PART 9 — SUBMISSION CHECKLIST

Complete every item before hitting submit on DoraHacks. Check each box.

#	Item	Notes
1	GitHub repo is PUBLIC	Settings → Danger Zone → Change visibility → Public
2	Apache 2.0 license file in repo root	Create LICENSE file with Apache 2.0 text from choosealicense.com
3	README.md explains: setup, architecture, how to run	Include: npm install, .env setup, npm run dev, working demo URL
4	All team members added to DoraHacks project page	Each teammate must confirm their DoraHacks account and be added
5	Video uploaded to YouTube as UNLISTED	Max 5 minutes. Paste the unlisted URL into DoraHacks submission
6	Working demo URL in submission	Your Vercel production URL. Judges must be able to access it directly
7	Track selected: Lending Bot	Select this as primary track in DoraHacks form
8	Third-party services disclosed	List: Alchemy, Anthropic Claude claude-haiku-3-5, Supabase, Ethereum Sepolia testnet, WDK by Tether
9	WDK is meaningfully integrated	Credit is confirmed — WDK creates wallets, sends USDT, monitors balances. Not just imported.
10	Agent autonomy demonstrated in video	Full cycle without human input must be visible on screen in the demo video